



@qa_insights

PANDAS

COMPLETE NOTES



Data Structures
& Series



DataFrames
& Operations



Data Cleaning
& Preparation



Data Analysis
& Aggregation



Visualization
& Insights

BEGINNER TO ADVANCED

SWIPE TO NEXT



Qa_insights | Better Careers

PANDAS FUNDAMENTALS

① What is Pandas?

Pandas is a powerful Python library used for data manipulation and analysis. It provides high-performance, easy-to-use data structures and tools to work with structured (tabular) data.

② Why use Pandas?

- ✓ Fast and efficient data handling
- ✓ Easy data cleaning and transformation
- ✓ Powerful data analysis capabilities
- ✓ Works with multiple data sources
- ✓ Rich ecosystem and integrations
- ✓ Widely used in industry and research

③ Pandas vs NumPy

Pandas	NumPy
Built for tabular (structured) data	Built for numerical arrays
Data structures: Series, DataFrame	Data structure: ndarray
Handles heterogeneous data easily	Best for homogeneous numeric data
High-level data manipulation tools	Low-level mathematical operations

④ Installing Pandas

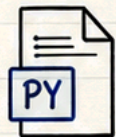
```
# Using pip
pip install pandas

# Using conda
conda install pandas
```



⑤ Importing Pandas

```
import pandas as pd
import numpy as np
```



⑥ Core Data Structures

Series (1D)

0	10	• One-dimensional • Labeled (indexed) • Can hold any data type
1	20	
2	30	
3	40	

Example: `pd.Series([10, 20, 30, 40])`

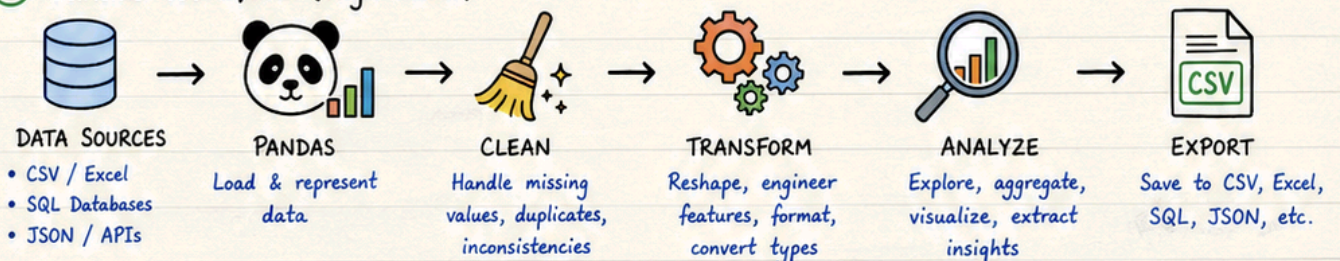
DataFrame (2D)

	ID	Name	Age	City
0	101	Alice	25	Delhi
1	102	Bob	30	Mumbai
2	103	Charlie	28	Bangalore

- Two-dimensional
- Rows & columns
- Heterogeneous data types

Example: `pd.DataFrame({...})`

⑦ Pandas Workflow (High-Level)



⑧ Takeaway



Pandas is Python's primary tool for working with structured/tabular data — from data loading to cleaning, transformation, analysis and export.

★ Remember

Pandas makes data work simple, fast and productive!

SERIES & DATAFRAME

CORE DATA STRUCTURES

① Series

- 1-dimensional labeled data structure.
- Consists of **Values** + **Index**.

→ Creating a Series

```
s = pd.Series([10, 20, 30])
```

→ Custom Indexes

```
s = pd.Series([10, 20, 30],
               index=['a', 'b', 'c'])
```

→ Selecting Values

```
s[0] # by position
# Output: 10
```

```
s.loc[0] # by label
# Output: 10
```

Series Example

Index	Values
0	10
1	20
2	30

← -- Values

← -- Index

② DataFrame

- 2-dimensional labeled data structure.
- Consists of **Rows** + **Columns** + **Index**.

→ Creating DataFrame

```
df = pd.DataFrame({
    "Name": ["A", "B"],
    "Age": [20, 25]
})
```

DataFrame Example

	Columns	
	Name	Age
Index -->	0	20
	1	25

← -- Data

③ Important Attributes

- **df.shape** - Returns (rows, columns)
- **df.columns** - Returns column labels
- **df.index** - Returns index labels
- **df.dtypes** - Returns data types of columns
- **df.values** - Returns underlying numpy array

```
df.shape
(2, 2)
```

```
df.columns
Index(['Name', 'Age'], dtype='object')
```

```
df.index
RangeIndex(start=0, stop=2, step=1)
```

```
df.dtypes
```

```
Name    object
Age      int64
dtype: object
```

```
df.values
```

```
array([[ 'A', 20],
       [ 'B', 25]], dtype=object)
```

④ From Series to DataFrame

Series (1D)

Values
10
20
30

(As Column)

	Values
0	10
1	20
2	30

(Multiple Columns)

	A	B
0	10	100
1	20	200
2	30	300

DataFrame (2D Table)

	Name	Age
0	A	20
1	B	25



TIP: Series is like a single labeled column.

DataFrame is like a spreadsheet or SQL table.

Remember:

Series → 1D labeled data

DataFrame → 2D labeled data



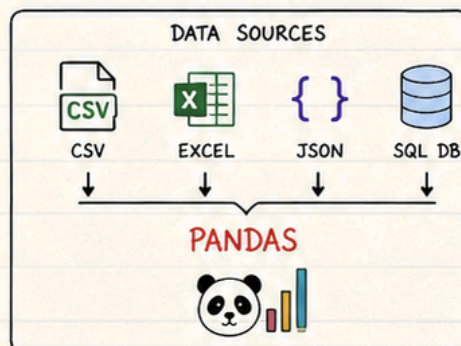
LOAD & EXPLORE DATA

READ + INSPECT DATA

© qa_insights

① Read Data (Import from various sources)

`pd.read_csv()` - Load data from CSV files
`pd.read_excel()` - Load data from Excel files
`pd.read_json()` - Load data from JSON files
`pd.read_sql()` - Load data from SQL databases



② First Look (Quick Preview)

`df.head()` • First 5 rows (default)
`df.tail()` • Last 5 rows (default)
`df.sample()` • Random sample of rows

Example:
`df.head()`

	ID	Name	Age	City	Sales
0	101	Alice	25	Delhi	5000
1	102	Bob	30	Mumbai	6500
2	103	Charlie	28	Bangalore	7000
3	104	David	32	Pune	4800
4	105	Eva	24	Chennai	5200
...	...	...	...	...	...

③ Understand Structure

`df.shape` • (rows, columns) of DataFrame
`df.info()` • Summary of dataframe (types, nulls)
`df.dtypes` • Data types of each column
`df.columns` • Column labels

Example Output

`df.shape`
(1000, 5)

`df.columns`

Index(['ID', 'Name', 'Age', 'City', 'Sales'], dtype='object')

`df.dtypes`

ID int64
Name object
Age int64
City object
Sales int64
dtype: object

`df.info()`

RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
Column Non-Null Count Dtype
0 ID 1000 non-null int64
1 Name 1000 non-null object
2 Age 1000 non-null int64
3 City 1000 non-null object
4 Sales 1000 non-null int64
dtypes: int64(3), object(2)
memory usage: 39.2+ KB

④ Statistical Overview

`df.describe()` • Summary statistics for numeric columns
`df.nunique()` • Count of unique values per column
`df.value_counts()` • Frequency count of unique values (useful for categorical)

`df.describe()`

	ID	Age	Sales
count	1000.0	1000.0	1000.0
mean	550.50	28.65	5780.45
std	288.82	5.60	1250.30
min	101.00	18.00	1500.00
25%	325.75	24.00	4900.00
50%	550.50	28.00	5600.00
75%	775.25	32.00	6500.00
max	1000.00	40.00	12000.00

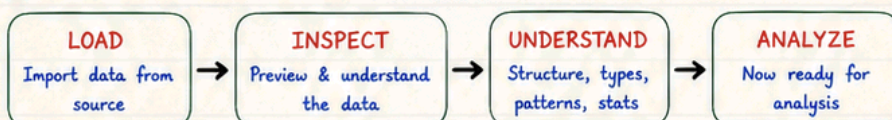
`df.nunique()`

ID	1000
Name	120
Age	23
City	6
Sales	450
dtype:	int64

`df['City'].value_counts()`

Mumbai	220
Delhi	200
Bangalore	180
Pune	150
Chennai	130
Hyderabad	120
dtype:	int64

⑤ Basic Inspection Mindset



★ IMPORTANT NOTE

Always inspect a dataset before cleaning or transforming it. It saves time and prevents incorrect results!



SELECT & FILTER DATA

① Select Columns

```
df["Age"]
```

- Single column

```
df[["Name", "Age"]]
```

- Multiple columns

Example DataFrame (df)

	ID	Name	Age	City	Salary
0	101	Alice	25	Delhi	5000
1	102	Bob	30	Mumbai	6000
2	103	Charlie	28	Bangalore	5500
3	104	David	32	Hyderabad	7000
4	105	Eva	24	Chennai	4500

② Select Rows → loc (by label)

```
df.loc[0]
```

- Row with label 0

```
df.loc[0:3]
```

- Rows with label 0 to 3 (inclusive)

```
df.loc[:, "Age"]
```

- All rows, column "Age"

df.loc[0]

ID	Name	Age	City	Salary
101	Alice	25	Delhi	5000

df.loc[0:3]

	ID	Name	Age	City	Salary
0	101	Alice	25	Delhi	5000
1	102	Bob	30	Mumbai	6000
2	103	Charlie	28	Bangalore	5500
3	104	David	32	Hyderabad	7000

df.loc[:, "Age"]

	Age
0	25
1	30
2	28
3	32
4	24

③ Select Rows → iloc (by integer position)

```
df.iloc[0]
```

- Row at position 0

```
df.iloc[0:3]
```

- Rows at positions 0 to 2

```
df.iloc[:, 1]
```

- All rows, column at position 1 ("Name" column)

df.iloc[0]

ID	Name	Age	City	Salary
101	Alice	25	Delhi	5000

df.iloc[0:3]

	ID	Name	Age	City	Salary
0	101	Alice	25	Delhi	5000
1	102	Bob	30	Mumbai	6000
2	103	Charlie	28	Bangalore	5500

df.iloc[:, 1]

	Name
0	Alice
1	Bob
2	Charlie
3	David
4	Eva

④ Conditional Filtering

```
df[df["Age"] > 25]
```

- Rows where Age > 25

	ID	Name	Age	City	Salary
1	102	Bob	30	Mumbai	6000
2	103	Charlie	28	Bangalore	5500
3	104	David	32	Hyderabad	7000

⑤ Multiple Conditions

```
df[(df["Age"] > 25) &
(df["City"] == "Hyderabad")]
```

- Rows where Age > 25
AND City is Hyderabad

	ID	Name	Age	City	Salary
3	104	David	32	Hyderabad	7000

⑥ Useful Filters

```
df["City"].isin(["Delhi", "Hyderabad"])
```

- City is Delhi OR Hyderabad

```
df["Age"].between(20, 30)
```

- Age between 20 and 30 (inclusive)

```
df.query("Age > 25")
```

- Age greater than 25 (using query)

Example Outputs

	ID	Name	Age	City	Salary
0	101	Alice	25	Delhi	5000
1	102	Bob	30	Mumbai	6000
2	103	Charlie	28	Bangalore	5500
3	104	David	32	Hyderabad	7000
4	105	Eva	24	Chennai	4500

isin → rows
0 and 3

between(20, 30)
→ rows
0, 1, 2, 4

query("Age > 25")
→ rows
1, 2, 3

★ Remember

loc → labels (index names)

iloc → integer positions



IMPORTANT NOTE

Always inspect a dataset before cleaning or transforming it. It saves time and prevents incorrect results!

DATA CLEANING

CLEANING REAL-WORLD DATA

① Missing Values

```
df.isna()
df.isnull()

df.dropna()
df.fillna(0)
```

- Detect missing values
- Drop rows/columns with missing values
- Fill missing values with a specified value

② Check Missing Values

```
df.isna().sum()
```

Returns count of missing values for each column

Column	Missing Count
ID	0
Name	2
Age	1
City	0
Salary	3

③ Duplicates

```
df.duplicated()
df.drop_duplicates()
```

- Detect duplicate rows
- Remove duplicate rows

Before				After		
ID	Name	Age	df.drop_duplicates() →	ID	Name	Age
1	A	25		1	A	25
2	B	30		2	B	30
2	B	30		3	C	28
3	C	28				

④ Data Types

```
df.astype(int)
pd.to_numeric(df['Age'])
pd.to_datetime(df['Date'])
```

- Change data type
- Convert to numeric
- Convert to datetime

Before (object)		Convert Types	After (int / datetime)	
Age	Date		Age	Date
"25"	"2024-01-01"		25	2024-01-01
"30"	"2024-01-02"		30	2024-01-02
"28"	"2024-01-03"		28	2024-01-03

⑤ Rename Columns

```
df.rename(
    columns={"old": "new"}
)
```

Rename one or multiple columns

Before

old_name	old_age	city
A	25	Delhi
B	30	Mumbai

→

After

Name	Age	city
A	25	Delhi
B	30	Mumbai

⑥ Text Cleaning (String Operations)

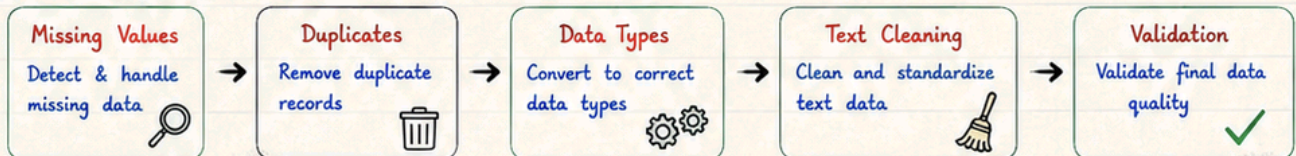
```
df["Name"].str.strip()
df["Name"].str.lower()
df["Name"].str.upper()
df["Name"].str.replace(" ", "-")
df["Name"].str.contains("A")
```

- Remove extra spaces
- Convert to lower case
- Convert to upper case
- Replace text
- Check pattern exists

Before	Cleaning Steps	After
Name		Name
" Alice "		alice
BOB		bob
Cha rlie		cha_rlie

★ **TIP**
String functions help clean text data before analysis.

⑦ Cleaning Pipeline (Recommended Order)



✓ Remember

Clean data → Reliable analysis → Better insights



IMPORTANT NOTE

Always clean your data before analysis.
Garbage in, garbage out!

TRANSFORM DATA

CREATE + MODIFY + TRANSFORM

① Create Columns

```
df["Total"] = (
    df["Price"] *
    df["Quantity"]
)
```

- Create a new column by performing operations on existing columns.

Before

Price	Quantity
100	2
250	3
120	5

After

Price	Quantity	Total
100	2	200
250	3	750
120	5	600

② Vectorized Operations

```
df["Salary"] * 1.10
```

- Perform operations on entire columns without loops.
- Faster and more efficient.

Example:

Salary	Salary + 10%
50000	55000.0
60000	66000.0
70000	77000.0

③ Apply Functions

```
df["Name"].apply(
    lambda x: x.upper()
)
```

- Apply a Python function to each element in a column.

Example:

Before

Name
alice
bob
charlie

After

Name
ALICE
BOB
CHARLIE

④ Conditional Values

```
np.where(
    df["Age"] >= 18,
    "Adult",
    "Minor"
)
```

- Return values based on a condition.
- Very useful for creating categorical columns.

Example:

Age	Category
16	Minor
18	Adult
25	Adult
12	Minor

⑤ Replace Values

```
df.replace({
    "Old": "New",
    "Yes": 1,
    "No": 0
})
```

- Replace one or more values in the DataFrame.
- Works for numbers, strings and more.

⑥ Sorting

```
df.sort_values("Salary")
df.sort_index()
```

- Sort rows by values in a column.
- Sort rows by index.

Sort by Salary

Name	Salary
Bob	50000
Alice	60000
Charlie	70000

Sort by Index

	Name	Salary
0	Alice	60000
1	Bob	50000
2	Charlie	70000

⑦ Add Multiple Columns (assign)

```
df.assign(
    Bonus=df["Salary"] * 0.10,
    NewSalary=df["Salary"] * 1.10
)
```

- Add one or more new columns in a single step.
- Returns a new DataFrame.

Example:

Name	Salary	Bonus (10%)	NewSalary (10%↑)
Alice	60000	6000.0	66000.0
Bob	50000	5000.0	55000.0
Charlie	70000	7000.0	77000.0

⑧ Golden Rule



Prefer **vectorized** operations over unnecessary loops. Better performance, cleaner code, and more Pythonic! ✓

Vectorized (Recommended)

```
df["Salary"] * 1.10
```

Fast, concise, efficient ✓

VS

Loop (Avoid)

```
new = []
for x in df["Salary"]:
    new.append(x * 1.10)
```

Slow, verbose, error-prone ✗

GROUPBY & AGGREGATION

SUMMARIZE DATA

① Basic GroupBy

```
df.groupby("Department")
```

- Groups rows based on unique values in the specified column.

Example DataFrame (df)

ID	Department	Employee	Salary
1	HR	Alice	50000
2	IT	Bob	60000
3	HR	Charlie	55000
4	IT	David	70000
5	Finance	Eva	65000

GroupBy (Split)

HR		
1	Alice	50000
3	Charlie	55000
IT		
2	Bob	60000
4	David	70000
Finance		
5	Eva	65000

② Aggregations

- `.sum()` - Sum of values
- `.mean()` - Average of values
- `.median()` - Median of values
- `.min()` - Minimum value
- `.max()` - Maximum value
- `.count()` - Count of values
- `.nunique()` - Count of unique values

③ Example

```
df.groupby(("Department"))["Salary"].mean()
```

Result:

Department	Salary (mean)
Finance	65000.0
HR	52500.0
IT	65000.0

④ Multiple Aggregations

```
df.groupby("Department")["Salary"].agg([
    "mean",
    "min",
    "max"
])
```

Result:

Department	Salary		
	mean	min	max
Finance	65000.0	65000	65000
HR	52500.0	50000	55000
IT	65000.0	60000	70000

⑤ Named Aggregation

```
df.groupby("Department").agg(
    avg_salary=("Salary", "mean"),
    employees=("ID", "count")
)
```

Result:

Department	avg_salary	employees
Finance	65000.0	1
HR	52500.0	2
IT	65000.0	2

⑥ Related Tools

- `df.pivot_table()`
Summarize data in a pivot table (multi-level aggregation).

Example Pivot

Department	Salary (mean)
HR	52500.0
IT	65000.0
Finance	65000.0

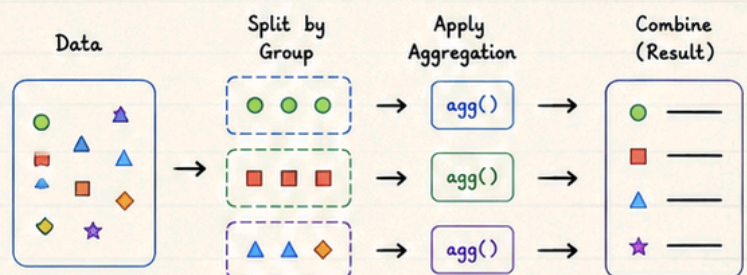
- `pd.crosstab()`
Compute frequency table of two or more factors.

Example Crosstab

City	Department		
	HR	IT	Finance
Delhi	2	3	1
Mumbai	1	2	1

⑦ Mental Model

SPLIT → APPLY → COMBINE



★ Remember

GroupBy allows you to split data into groups, apply functions, and get summarized results.



IMPORTANT NOTE

GroupBy operations exclude missing (NaN) values in the grouped column by default.

MERGE, JOIN & CONCAT

COMBINE DATASETS

① MERGE

```
pd.merge(
    df1,
    df2,
    on="ID"
)
```

- Combines two DataFrames using a common column (key).
- Similar to SQL JOIN operation.

Example (ON = "ID")

df1

ID	Name	Dept
1	Alice	HR
2	Bob	IT
3	Charlie	Finance
4	David	HR

df2

ID	Salary
2	60000
3	55000
5	70000

INNER JOIN

ID	Name	Dept	Salary
2	Bob	IT	60000
3	Charlie	Finance	55000

LEFT JOIN

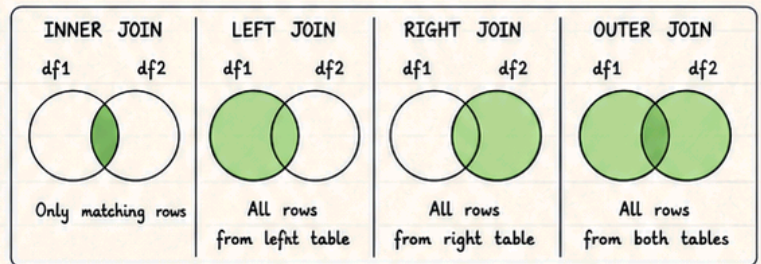
ID	Name	Dept	Salary
1	Alice	HR	NaN
2	Bob	IT	60000
3	Charlie	Finance	55000
4	David	HR	NaN

RIGHT JOIN

ID	Name	Dept	Salary
2	Bob	IT	60000
3	Charlie	Finance	55000
5	NaN	NaN	70000

OUTER JOIN

ID	Name	Dept	Salary
1	Alice	HR	NaN
2	Bob	IT	60000
3	Charlie	Finance	55000
4	David	HR	NaN
5	NaN	NaN	70000



③ JOIN

```
df1.join(df2)
```

- Combines DataFrames using index (by default).
- Similar to LEFT JOIN.

Example (Index based)

df1

	ID	Name
0	101	Alice
1	102	Bob
2	103	Charlie

df2

	Dept
0	HR
1	IT
3	Finance

df1.join(df2)

	ID	Name	Dept
0	101	Alice	HR
1	102	Bob	IT
2	103	Charlie	NaN

④ CONCATENATE

```
pd.concat(
    [df1, df2]
)
```

- Stack DataFrames together along an axis.
- Does NOT match on keys.

Example

df1

	ID	Name
0	101	Alice
1	102	Bob

df2

	ID	Name
2	103	Charlie
3	104	David

Row-wise (axis=0)

pd.concat([df1, df2], axis=0)

	ID	Name
0	101	Alice
1	102	Bob
2	103	Charlie
3	104	David

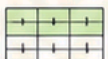
Column-wise (axis=1)

pd.concat([df1, df2], axis=1)

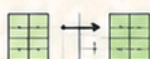
	ID	Name	ID	Name
0	101	Alice	103	Charlie
1	102	Bob	104	David

⑤ TWO DIRECTIONS

Rows (Stack vertically)
axis = 0 (default)



Columns (Stack horizontally)
axis = 1



⑥ CRITICAL DISTINCTION

- **MERGE** → Match data using keys (columns).
- **JOIN** → Combine using index/key relationship.
- **CONCAT** → Stack data together (no matching).

★ TIP

Use **MERGE** for relational data, **JOIN** for index-based alignment, and **CONCAT** for stacking data.

TIME SERIES + ADVANCED PANDAS

TIME + ADVANCED OPERATIONS

① Convert Dates

```
df["Date"] = pd.to_datetime(
    df["Date"]
)
```

- Converts string/object column to datetime format.

Before		After	
Date	Sales	Date	Sales
01-01-2024	100	2024-01-01	100
02-01-2024	150	2024-01-02	150
03-01-2024	120	2024-01-03	120

② Extract Date Components

```
df["Date"].dt.year
df["Date"].dt.month
df["Date"].dt.day
df["Date"].dt.day_name()
```

- Access useful parts of a datetime column.

Example Output

Date	year	month	day	day_name
2024-01-01	2024	1	1	Monday
2024-01-02	2024	1	2	Tuesday
2024-01-03	2024	1	3	Wednesday

③ Time Operations

```
df.resample("D").sum()
df.shift(1)
df.rolling(3).mean()
```

- resample()**: Change frequency & aggregate
- shift()**: Move data up/down
- rolling()**: Window based calculation

Resample (Daily Sum)

Date	Sales
2024-01-01	100
2024-01-02	150
2024-01-02	50
2024-01-03	120

Shift(1)

Sales
100
150
120
NaN

Rolling(3) Mean

Mean(3)
NaN
NaN
123.33
133.33

④ Transformations

```
df.apply(func)
df.map(dict_or_func)
df.transform(func)
df.assign(new_col=...)
```

- apply()**: Apply custom function on axis/column
- map()**: Element-wise map for Series
- transform()**: Return same as input
- assign()**: Add new columns

Mini Examples

apply()

Name	Upper
alice	ALICE
bob	BOB
charlie	CHARLIE

map()

City	Code
delhi	DL
mumbai	MH
chennai	TN

transform()

Sales	Rank
100	2
150	1
100	2

assign()

Sales	Sales	Bonus(10%)
100	100	10.0
150	150	15.0
120	120	12.0

⑤ Reshaping

```
df.melt(id_vars=["ID"],
        value_vars=["Jan", "Feb"],
        var_name="Month",
        value_name="Sales")
df.stack()
df.unstack()
```

- melt()**: Wide → Long format
- stack()**: Pivot columns into rows
- unstack()**: Pivot rows into columns.

Wide Format

ID	Jan	Feb	Mar
1	10	20	30
2	15	25	35

Long Format

ID	Month	Sales
1	Jan	10
1	Feb	20
1	Mar	30
2	Jan	15
2	Feb	25
2	Mar	35

stack()

ID	Month	Sales
1	Jan	10
1	Feb	20
1	Mar	30
2	Jan	15
2	Feb	25
2	Mar	35

unstack()

ID	Jan	Feb	Mar
1	10	20	30
2	15	25	35

⑥ MultiIndex

- Hierarchical indexing
- Multiple levels of row/column labels
- Useful for advanced grouping & reshaping

Row MultiIndex Example

		ID	Name	Sales
Year	Quarter			
2024	Q1	101	Alice	500
	Q2	102	Bob	600
	Q3	103	Charlie	700
	Q1	104	David	800

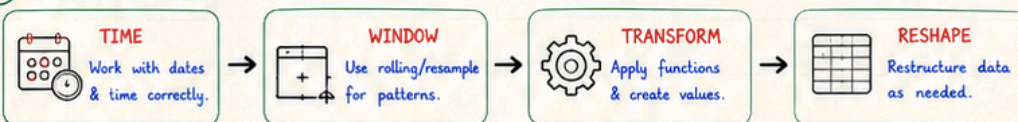
Column MultiIndex Example

ID	Sales		Profit	
	Online	Store	Online	Store
101	500	300	120	80
102	600	400	150	100

★ TIP

MultiIndex helps in handling complex hierarchical data combine time handling, transformations & reshaping smartly.

⑦ Advanced Mindset

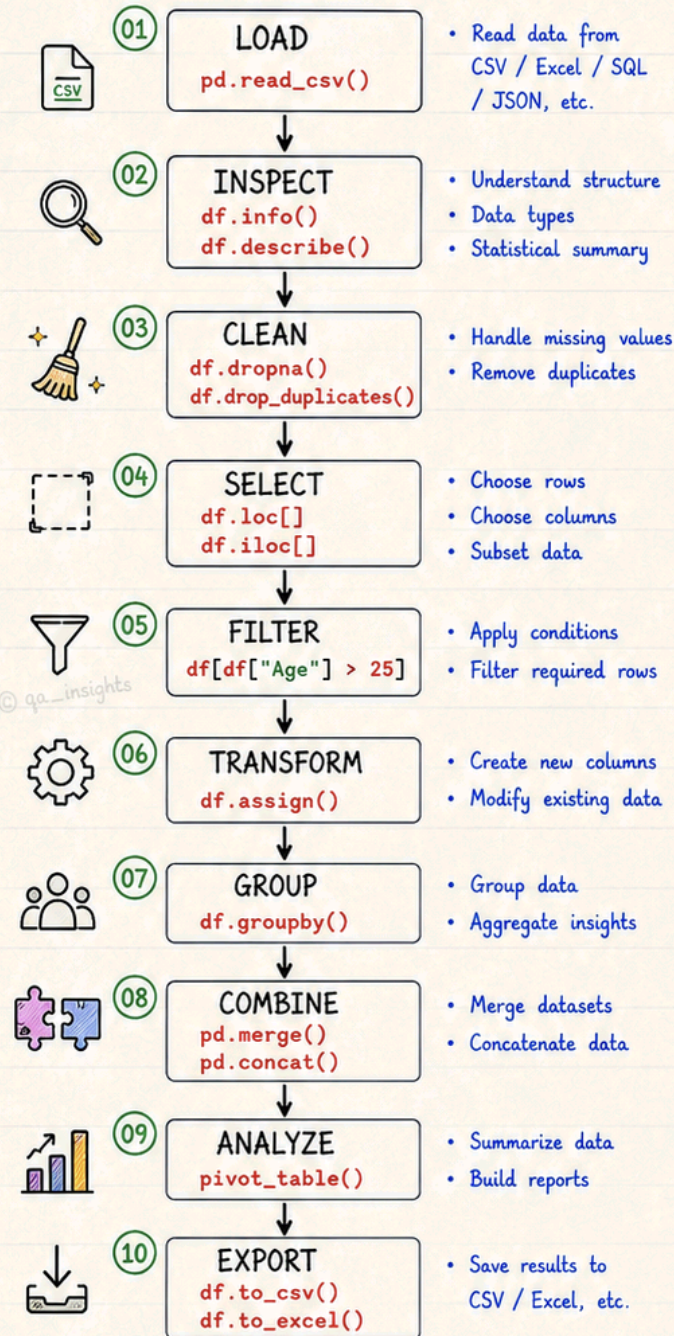


★ REMEMBER

Pandas becomes powerful when you combine time handling, transformations & reshaping smartly.

PANDAS MASTER WORKFLOW

END-TO-END WORKFLOW



PANDAS QUICK CHEAT SHEET

READ 	• <code>read_csv()</code> Read data into DataFrame
INSPECT 	• <code>head()</code> Preview data • <code>info()</code> DataFrame info • <code>describe()</code> Statistical summary
SELECT 	• <code>loc[]</code> Select by labels • <code>iloc[]</code> Select by integer position
CLEAN 	• <code>isna()/isnull()</code> Check missing values • <code>fillna()</code> Fill missing values • <code>dropna()</code> Drop missing values • <code>drop_duplicates()</code> Remove duplicate rows
TRANSFORM 	• <code>apply()</code> Apply custom function • <code>assign()</code> Add new columns • <code>replace()</code> Replace values
ANALYZE 	• <code>groupby()</code> Group data • <code>agg()</code> Aggregate functions • <code>pivot_table()</code> Pivot / summarize data
COMBINE 	• <code>merge()</code> Merge on keys • <code>join()</code> Join on index/key • <code>concat()</code> Concatenate data
RESHAPE 	• <code>melt()</code> Wide to long • <code>stack()</code> Pivot columns to rows • <code>unstack()</code> Pivot rows to columns
EXPORT 	• <code>to_csv()</code> Export to CSV • <code>to_excel()</code> Export to Excel

✓ FINAL TAKEAWAY

Load → Inspect → Clean → Select → Filter → Transform → Group → Combine → Analyze → Export

Master this workflow to handle any real-world data with confidence!

★ REMEMBER

- ✓ Always inspect your data before transforming.
- ✓ Clean data + Right operations = Better insights!